Concurrent Hash Tables and Radix Trees Under LLM-Derived Workloads

Nathan Lemma Joshua Chikosha Yonatan Teshome
EE 361C: Multicore Algorithms, Spring 2026

April 21, 2026

Abstract

LLM serving engines rely on prefix caching; the cache index must sustain many concurrent lookups without becoming a bottleneck. Production systems disagree on structure: vLLM uses hash tables, SGLang uses a radix tree, and trade-offs are still debated. We ask: (1) do typical concurrent hash-table rankings still hold under an LLM-derived workload; and (2) when prefix sharing is high, can a radix tree match or beat the best hash table?

We implement five concurrent hash-table variants and two radix-tree variants behind one interface and evaluate them on synthetic workloads, a ShareGPT-derived trace, and a tunable shared-prefix workload. **Workload shape changes the winner.** On uniform and Zipfian read-heavy mixes, `cuckoo_optimistic` leads; on the ShareGPT trace, `chaining_fine` overtakes it once write pressure exposes cuckoo's global displacement path. Concurrency control matters as much as the data structure: a coarse-lock radix anti-scales from 1 to 8 threads, while a CAS-based fine radix with lock-free internal traversal scales 3.81× over that range and matches or beats the best hash table when sharing is high. Synthetic benchmarks alone can mislead prefix-cache index design; both workload structure and synchronization strategy must be considered together.

1 Introduction

Deployed machine-learning systems spend a large fraction of operational resources on inference, not training; serving efficiency is no longer optional. Google reports a roughly 60/40 inference-to-training split in its ML energy use [1], and AWS notes that inference can account for a large share of SageMaker-related infrastructure cost [2].

To serve a Large Language Model (LLM) in production, the engine tokenizes each request into a sequence of *tokens* (subword units from a vocabulary, not raw characters alone). Surprisingly, many of the requests that come in every day start with a few tokens that are highly overlapping with other requests, with other parts of the serving stack, with system prompts, with few shot examples, with the beginning of a conversation, etc. In order to take advantage of these overlaps, the serving engine will cache the intermediate key-value (KV) computation for prefixes of requests so that subsequent requests with overlapping prefixes can skip re-computing the overlap. These caches are very large and dynamically growing, are stored under significant memory pressure, and have been known to run out of capacity and evict existing entries.

Prefix caching will only help if the engine can first determine whether a cached prefix matches the request path. This requires a concurrent data-structure lookup, since many threads will issue this same lookup simultaneously, and any non-thread-safe behavior in the index will be visible in request latency. The cache index is the object whose behavior must be considered when deciding whether prefix caching is beneficial under load.

Most comparisons of concurrent hash tables have been run under a variety of workloads, but almost always under uniform or Zipfian synthetic workloads. In this paper, we contrast the performance of several common concurrent hash tables under a different distribution of workload: prefix caching. Caching a prefix of a key does not typically have a uniform distribution of reuse; popular prefixes are used more than less popular ones. Furthermore, even if the hash function is uniform, most keys will have never been seen before, leading to bursts of inserts for previously unseen suffixes. In addition, many cache implementations use an LRU-style eviction policy, meaning that removals are also performed based on the recent reuse of keys. Therefore, a hash table that may perform well in a read-heavy workload may have a very different write path when prefix caching is used.

Production systems make different index choices. vLLM [3] uses block-level hash-table indexing for KV blocks; SGLang's RadixAttention [4] uses a radix tree that exploits shared prefix structure. Hash tables treat keys as flat objects; radix trees pay pointer-walk overhead but can amortize it when prefixes overlap. Which wins depends on both workload structure and how concurrent operations synchronize.

We evaluate the ranking of popular concurrent hash tables under uniform and Zipfian workloads and compare it to the prefix-cache workload distribution for Large Language Models (LLMs). In the second part of the paper, we study a concurrent radix tree data structure for workloads with high prefix-sharing, and evaluate whether it outperforms the best hash-table implementation or even surpasses their performance.

How to evaluate all of them? We present in this paper five concurrent hash tables (`chaining_coarse`,

Variant	Structure	Reader sync.	Writer sync.	Expected bottleneck
`chaining_coarse`	chained hash table	global mutex	global mutex	single lock
`chaining_fine`	chained hash table	per-bucket mutex	per-bucket mutex	hot-bucket contention
`cuckoo_striped`	bucketized cuckoo	stripe lock (64)	all-stripe lock on displacement	displacement serialization
`cuckoo_optimistic`	bucketized cuckoo	version snapshot (lock-free)	per-bucket lock; global mutex for displacement	global displacement mutex
`hopscotch`	hopscotch (H=32)	seqlock snapshot (lock-free)	global writer mutex	writer queue
`radix_tree_coarse`	256-way trie	global mutex	global mutex	single lock
`radix_tree_fine`	256-way trie	lock-free acquire-load walk	CAS on child slot; per-leaf mutex	per-leaf contention

Table 1: Synchronization strategy and expected bottleneck for each variant.

`chaining_fine`, `cuckoo_striped`, `cuckoo_optimistic`, and `hopscotch`) and two concurrent radix trees (`radix_tree_coarse` and `radix_tree_fine`) all evaluated through the same abstract data structure interface. We provide an empirical study of all of the concurrent data structures on both synthetic and real workloads: uniform and Zipfian insertion loads, loads with varying read to write ratios, cache-regime sweeps of a random mix, a real-world ShareGPT-derived trace that not only captures access patterns and eviction dynamics, and a tunable shared-prefix workload that isolates the benefits of a trie. We evaluate each hash table and radix tree in terms of throughput and tail latency for varying numbers of threads, as well as varying degree of data sharing.

Workload distribution shapes the winner, and synchronization discipline is as important as the underlying data structure. On uniform and Zipfian read-heavy mixes, `cuckoo_optimistic` leads `chaining_fine`; on the LLM-derived trace, `chaining_fine` overtakes it once write pressure makes cuckoo's displacement path visible. By keeping the radix structure constant and varying concurrency strategy alone, we achieve an increase in throughput of greater than an order of magnitude at 8 threads. A fine-grained radix structure can match or beat a high-quality hash table when there is high prefix sharing, but poor performance is observed when there is low sharing due to a lack of top-level locality. Synthetic workloads can be deceiving when selecting the prefix-cache index; as such, structure and synchronization must be considered together rather than in isolation.

2 Background and Motivation

2.1 Prefix caching

For each LLM inference request, token by token, and for each token position, we compute per layer keys and values (KV). Many of the requests will have some prefix of identical early tokens. We cache the per-block KV state for recent prefixes in a prefix cache, so that a later request can use that cached computation for the identical tokens as opposed to recomputing attention. We fill the cache as we see prefixes, query it for every request, and then shrink the cache down when under memory pressure with an eviction policy.

2.2 Why cache organization matters

Since GPU memory is limited, each KV block is not cheap. As a result, the serving engine has to address two challenges at the same time: (1) efficient in-memory storage of the prefix state, and (2) efficient location of cache entries that can be reused. This paper focuses on the second challenge as a data structure problem. If location of cache entries took too long or was serial, caching even with a large amount of memory allocated to caching would be inefficient.

2.3 A concurrent data-structure problem

The cache index is located on the request path, and therefore must be safely queried by multiple worker threads concurrently. The workload is read-heavy, but mostly lookups for an arriving prefix, as opposed to expensive writes that form new prefixes by adding suffixes. While some entries will be evicted when they reach cold lifetime and replaced by others, this kind of reuse plus eviction churn can strongly dominate the behavior of an index under synthetic read-heavy workloads.

2.4 Hash tables versus radix trees

Hash tables that treat keys as objects (flat data structures) have efficient concurrent implementations that support constant time lookups. Radix trees on the other hand follow a shared prefix path of keys with repeated usage, which can amortize the cost of node accesses for nearby prefixes in concurrent implementations. Whether either helps in practice, however, depends on the degree of sharing (i.e., the number of keys that share prefixes) and whether the shared prefix was recently used. Even if both prefix sharing and the resulting improvements in work and cache performance are valuable, additional synchronization overhead on top of the tree data structure may prevent such benefits.

3 Design and Implementation

All data structure variants are extended to support `insert`, `lookup`, and `remove` operations for 64-bit keys and values. The tables are fixed size — they do not resize. The same testing harness is used for benchmarking and the same correctness testing matrix is applied for all of the variants. The purpose of this paper is to perform a controlled comparison of data structures and synchro-

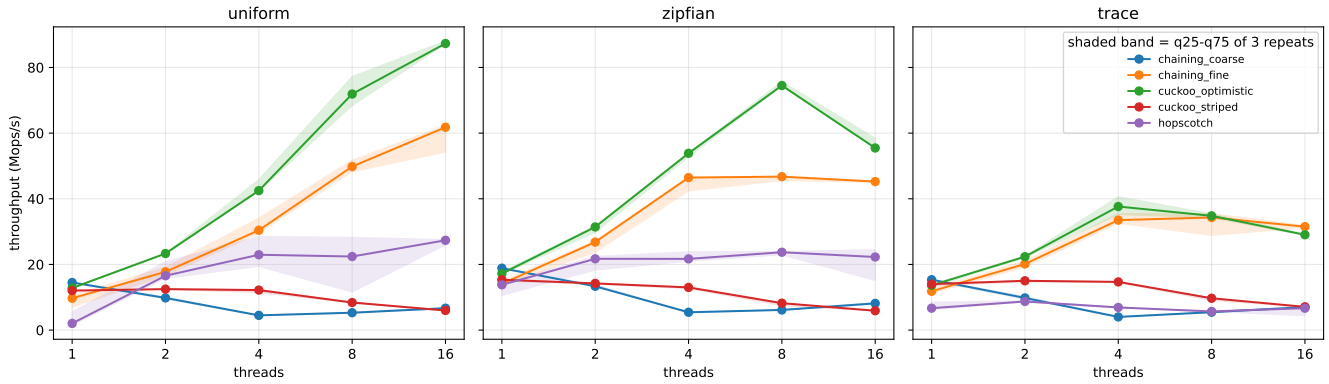

Figure 1: Throughput vs. threads for five hash-table variants on uniform, Zipfian, and ShareGPT-trace workloads at 95% reads (median of three repeats, q25–q75 band). Single-lock designs sit below ∼10 Mops/s. `cuckoo_optimistic` leads `chaining_fine` by 1.41× on uniform but is overtaken on the trace (0.92×) as the ∼39.6% write fraction loads cuckoo's global displacement mutex.

nization methods on a well defined battlefield — not to write a production-complete implementation. Table **??** summarizes the synchronization strategy and bottleneck for each variant.

3.1 Hash-table variants

The five hash designs span four synchronization regimes: coarse-grained locking, fine-grained locking, optimistic reads over versioned state, and explicitly serialized writers. For each variant we summarize structure, how readers and writers synchronize, and the expected bottleneck.

`chaining_coarse` is a separate chaining hash table protected by a single global mutex. Both readers and writers of the `chaining_coarse` hash table serialize through the same lock. The ceiling of the locking mechanism is single-writer throughput.

In the `chaining_fine` hash table, each bucket locks with a separate mutex. This mutex is cache-line aligned on the metadata so you do not get false sharing between buckets. So only the bucket that a reader or writer is processing at any given time should ever be locked, which means you primarily contend on hash skewness between buckets plus eviction contention on the hot buckets.

`cuckoo_striped` is an implementation of bucketized cuckoo hashing with 64 striped locks. Lookups lock a single stripe while inserts that displace hash will lock every stripe for a full rewrite, so the displacement path behaves like a global lock under write pressure.

`cuckoo_optimistic` is bucketized cuckoo (four slots per bucket, two hashes) with per-bucket version counters and per-bucket writer locks. Readers take no locks on the fast path: they snapshot versions, read key/value pairs, and revalidate before accepting a read. Writers first try both home buckets; on failure they take a table-global mutex and run a conservative one-hop displacement (a victim is evicted only when its alternate slot is empty). Under sustained inserts the global displacement mutex is the expected bottleneck.

`hopscotch` is optimistic hopscotch with H=32 neighborhoods. Each bucket carries an atomic `key/value`, a 32-bit `hop_info` bitmap, and a seqlock version counter. Readers scan only the slots named by `hop_info` and revalidate the version. All writers serialize on one global mutex to keep the bubble-up invariant simple; the expected bottleneck is the writer queue.

We present five distinct hash table variants, allowing comparative studies of concurrent design choices. Two of the variants, `cuckoo_optimistic` and `hopscotch`, have a serialized write path, on displacement for cuckoo and on every insert or remove for hopscotch. The word "optimistic" in `cuckoo_optimistic` refers only to the lock-free reader path and not to the writer path. As a consequence, even for data structures with write-heavy or churn-heavy workloads, lock-free readers are not sufficient to avoid writer stalls.

3.2 Radix-tree variants

Unlike the hash-table comparison, the radix comparison isolates concurrency discipline on a fixed trie shape. Both variants are byte-level 256-way tries over 64-bit keys; they differ only in coordination.

`radix_tree_coarse` protects the entire trie with one `std::mutex`, so throughput differences from the fine variant are due to synchronization, not structure.

`radix_tree_fine` replaces the global mutex with a lock-free descent: each node's `children[256]` slots are atomic pointers; readers and writers acquire-load down the tree. A writer installs a new child with `compare_exchange_strong` on the parent slot; if it loses the race, it `deletes` its candidate and follows the winner's pointer. A per-leaf `std::mutex` guards `has_value` and `value` so leaf reads are not torn. Safety relies on never freeing nodes during concurrent operation: removal clears `has_value` but does not unlink nodes; deallocation runs only in the destructor after threads join. This avoids SMR/RCU/hazard pointers but allows monotonic

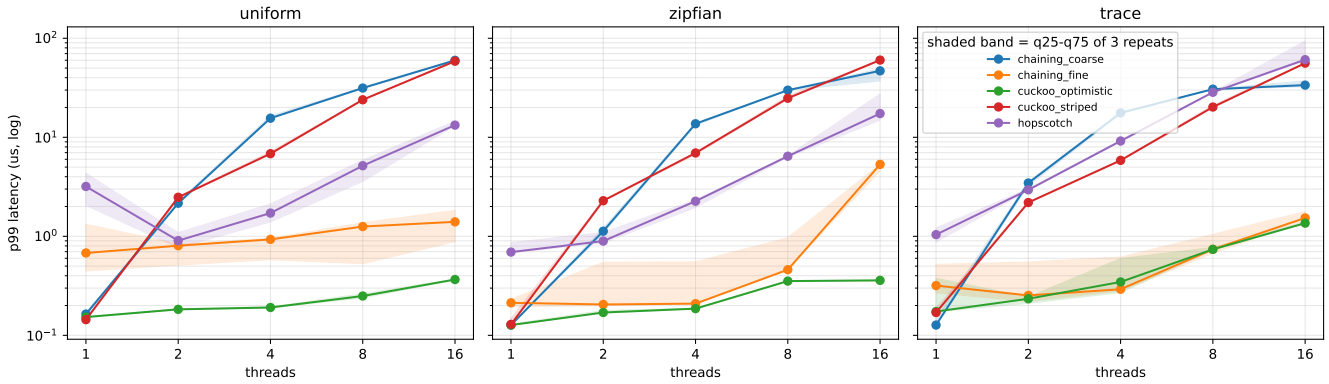

Figure 2: p99 latency vs. threads for the hash-table variants (95% reads, median of three repeats). Optimistic-reader designs stay sub-microsecond on synthetics; `hopscotch`'s writer mutex inflates p99 to ∼60 μs on the trace.

memory growth over a run.

3.3 Design rationale

We also explored an intermediate approach that used hand-over-hand locking with `std::shared_mutex` per node. However, this ran roughly 8× slower than `radix_tree_coarse` at 8 threads (running on macOS, compiled with libc++). Hand-over-hand locking with plain `std::mutex` per node ran roughly 13× slower than `radix_tree_coarse`. This was based on the root mutex locking all operations, and then per-node locks locking within the node that was currently locked. What we call lock-free walks removes the root mutex lock entirely, except in the relatively rare path where a single CAS installs a new child node. However, contention can still occur across the per-leaf mutex when multiple threads traverse the tree and operate on the same value.

4 Workloads and Experimental Setup

4.1 Workloads

We study hash-table and radix performance using three workload families: an LLM-derived trace with strong reuse and eviction churn; standard synthetic mixes used for comparability in the literature; and a tunable shared-prefix workload that isolates the effect of prefix sharing on trie behavior.

4.1.1 LLM-derived trace

We build `data/sharegpt.trace` from the `RyokoAI/ShareGPT52K` conversation dataset using `tiktoken` with the `cl100k_base` encoder. Text is segmented into 16-token blocks; each block corresponds to one logical cache lookup (and an insert if that block has not been seen before). Each hit to the cache can then replay on the order of N lookups to the cache as the prefix cache is reused. We use a bounded LRU policy when the cache cap is hit. All of the data in this paper was generated from a single canonical trace, which was set up with an LRU cap of 100,000 blocks and a replay factor of N=2. The resulting trace has 1,000,000 lookups, inserts, and removes in total, 357,748 unique keys, with 60.45% lookups, 35.78% inserts, and 3.78% removes. This trace v2 binary header format includes provenance, dataset ID, encoder, block size, LRU cap, and a timestamp (in seconds since the epoch). For the post-warmup replay, the read fraction came out to be about 0.604.

This workload combines skewed reuse (not pure Zipfian imagination), bursts of inserts for new suffixes, and LRU-driven removes — so both read and write paths of the index matter, not just lookup rate under a static distribution.

4.1.2 Synthetic baselines

For all benchmarks, we use four different mixes: uniform access, Zipfian access (α=0.99), a sweep of read-fraction at 8 threads with read fractions $\{1.0, 0.95, 0.5, 0.1\}$ lookups, and a sweep of cache regimes (uniform, 95% reads, 100K key universe). We do two thread sweeps on the uniform mix and the Zipfian mix at 95% reads. This allows us to test the concurrent hash tables and understand performance differences under the workloads for which we compare implementations; and to determine if the performance leader changes for reasons unrelated to the implementation details under consideration.

4.1.3 Shared-prefix workload

To demonstrate the radix variants, we have created a synthetic shared-prefix workload. The keys are eight bytes long, with each key being a 64-bit integer split into a 32-bit `prefix_id` and a 32-bit `suffix`. `prefix_id` is uniformly distributed over the range $[0, P)$, and `suffix` is uniformly distributed over the range $[0, U/P)$ for some total number of unique keys U. We keep U constant and vary P (the average sharing factor) with the `-prefix-count` option. This workload can be heavy on top-level sharing in the trie, and we can step all the way down to having nearly unique prefixes. We apply the same distribution when selecting the small ranges to prepopulate, so lookups that are nominally distant in the trie can be hot in this workload. This allows us

to separately observe the benefits of the radix tree data structure for a fixed universe, and study the effects of sharing of prefixes.

4.2 Experimental setup

4.2.1 Platform

All experiments were run on one Apple M4 Pro chip (14 CPU cores) with 24 GB of RAM. The C++ code was compiled with Apple clang 21.0.0. For the thread sweep experiments, we used $\{1, 2, 4, 8, 16\}$ threads. For the radix experiments, we used 1 and 8 threads. macOS 26.4 (Darwin 25.4.0) was the operating system. Valgrind was not used in any of these experiments.

4.2.2 Measurement methodology

Each configuration is run three times and results tabulated. We run each configuration three times with different seeds (`seed = base + repeat_idx`) and each row of the resulting CSV is effectively a single data point `(seed, repeat, git_sha)`. We use `ops_per_run = 500,000` and `warmup_ops_per_thread = 5,000`. All throughputs and p99s are the median of three runs with a shaded q25–q75 region.

4.2.3 Latency measurement

This latency data is a two-zone histogram: 64 ns linear bins to 16 μs, and then log2 bins after 16 μs. p50, p99, and p999 are linearly interpolated within the occupied bin. `max_ns` is the true sampled maximum. The clock on a macOS machine has a floor of 40–100 ns, so you will not be able to resolve latency differences that are below that level. Each row of latency data includes `insert_attempts` and `insert_successes`, so failed inserts are visible rather than silent.

4.2.4 Correctness validation

The canonical test for correctness is `correctness_test -threads=16 -repeats=10`. Every table runs single-threaded sanity tests, then concurrently inserting into a table, and then concurrently mixing read and write operations on a table. Optimistic hash tables also run `displacement_stress`, `inserter_lookup_race`, and `multi_writer_displacement` in addition to the above tests. Hopscotch hash tables run an additional `hopscotch_bubble_race` test. We recently added a read-heavy version of the mixed test (16 threads, 90% lookups and 10% inserts/writes) to a prepopulated table that is shared between threads. This allows us to stress test the lock-free radix walk. It does 720K lookups at a time, and concurrently does 80K inserts/writes. Any hit that returns a value that is not correct is a torn-read failure. We have a full matrix of table types versus these test types, and all tables pass all the tests in the matrix. ThreadSanitizer can be run on the fully lock-based equivalent of lock-free algorithms. It reports the optimistic-reader pattern and the lock-free radix walk as races by construction.

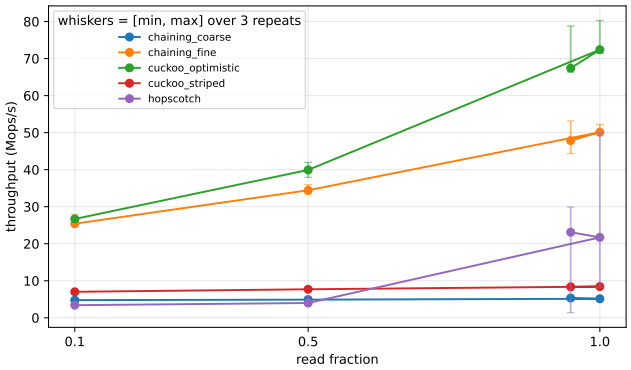

Figure 3: Throughput vs. read fraction at 8 threads, uniform, 10M keys. `hopscotch` is the most read-sensitive design: writer-mutex contention dominates whenever writes are present.

5 Evaluation

This evaluation first applies the hash-table story to compare hash tables under standard synthetic workloads, then inverts the workload to compare performance using an LLM-derived trace, follows that by studying tail latency and workload sensitivity, and finally compares radix trees under two axes: concurrency and structure (fixed implementation, varying design) and sharing and size (fixed structure, varying sharing and scale).

5.1 Hash tables under standard synthetic workloads

Figure 1 shows the thread sweep for all competing implementations. On the two synthetic data mixes, both tests again show consistent ordering. On uniform / 95% reads, `cuckoo_optimistic` led with 87.30 Mops/s (q25–q75 87.10–88.78), followed by `chaining_fine` at 61.79 Mops/s (46.70–62.87), then `hopscotch` at 27.36 (24.51–27.72), followed by `chaining_coarse` at 6.67 Mops/s and `cuckoo_striped` at 5.98 Mops/s. On Zipfian / 95% reads, all designs decreased in absolute throughput, but again maintained the same ordering: `cuckoo_optimistic` 55.47 Mops/s, `chaining_fine` 45.23 Mops/s, `hopscotch` 22.27, with the single-lock designs clustered at 5.9–8.1 Mops/s. Optimistic cuckoo's versioned reader path is the clear winner on read-heavy workloads. Fine chaining was competitive, and every single-lock or full-stripe-lock design failed to saturate at less than 10 Mops/s.

5.2 The LLM-derived trace changes the ranking

For the ShareGPT trace, the same sweep inverts the top two. At 16 threads, `chaining_fine` reaches 31.51 Mops/s (30.90–32.14) and overtakes `cuckoo_optimistic` at 29.04 (28.69–29.42); single-lock designs and `hopscotch` sit near 6.7–7.1 Mops/s. The trace carries $\sim$ 39.6% writes (35.78% inserts + 3.78% removes), which repeatedly activates `cuckoo_optimistic`'s global displacement mutex; `chaining_fine` has no equivalent global path on its read-mostly lookups. `hopscotch` stalls on its single writer

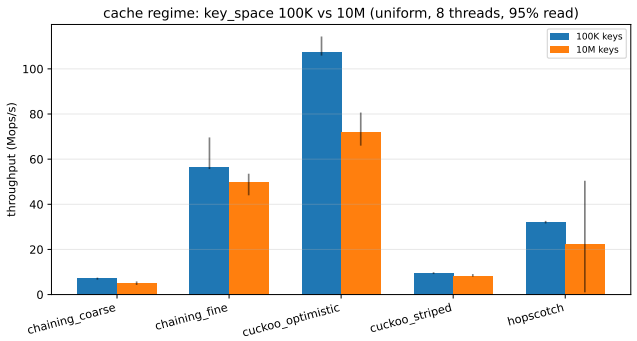

Figure 4: Throughput under a cache-resident working set (uniform, 8 threads, 95% reads, `key_space=100K`). Optimistic cuckoo's reader path benefits the most from L2 residency.

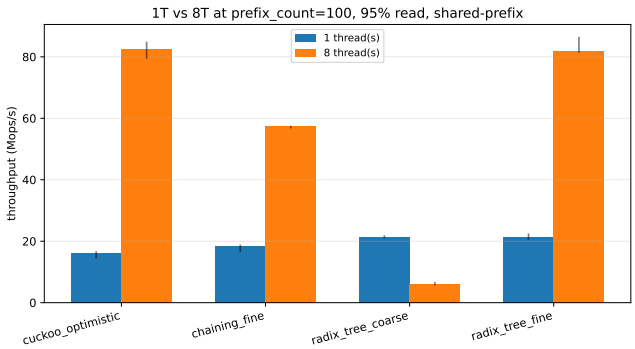

Figure 5: Throughput per table for 1-thread vs. 8-thread. `prefix_count`=100, 95% reads, shared-prefix. `radix_tree_coarse` anti-scales from 21.48 to 6.08 Mops/s (global-mutex queue) whereas `radix_tree_fine` scales 3.81× from 21.46 to 81.86 Mops/s. Coarse-to-fine increase in throughput for the same data structure but different concurrency policy is 13.47×.

mutex for every insert and remove. Rankings from uniform and Zipfian benchmarks alone do not predict this behavior under real prefix-cache churn.

5.3 Tail latency

Throughput is not the whole story. Figure 2 shows p99 at 16 threads. On synthetic mixes `cuckoo_optimistic` has the lowest p99 (0.37 / 0.36 μs on uniform / Zipfian) and is competitive on the trace (1.36 μs). `chaining_fine` reports 1.40 / 5.32 / 1.53 μs across uniform, Zipfian, and trace, and slightly edges `cuckoo_optimistic` on trace p99 despite a higher Zipfian tail. `hopscotch`'s p99 is 13.28 / 17.37 μs on synthetic and 60.84 μs on the trace — writer-queue delay under write pressure. Single-lock designs stay at 34–60 μs throughout. Where throughput highlights optimistic readers, tail latency highlights writer serialization.

5.4 Read-fraction sensitivity

We also ran the uniform / 10M keys workload at 8 threads, increasing the read fraction and measuring performance (Figure 3). `cuckoo_optimistic` performance increases fairly linearly to 72.40 Mops/s (from 26.69 Mops/s at 0.1 reads) at 1.0 reads. `chaining_fine` increases more read-sensitively to 50.11 Mops/s (from 25.41 Mops/s at 0.1 reads) and plateaus at around 0.95 reads. `hopscotch` is the most read-sensitive design and increases most dramatically to 23.11 (from 3.40 Mops/s at 0.1 reads) at 0.95 reads and 21.71 Mops/s at 1.0 reads. The pure-read cell is held back by its single atomic writer path. `chaining_coarse` and `cuckoo_striped` do not scale to 8 threads and therefore their numbers are flat. The shape of the sensitivity curves for optimistic cuckoo versus fine-grained chaining is what we expected: optimistic cuckoo's lead over fine-grained chaining decreases faster as write rate increases, and that is exactly what the trace observed.

5.5 Cache-resident regime

Reducing the key universe size is a much more effective operation for improving cache performance for some data structures, but not for others (Figure 4). While `chaining_fine` still increases throughput, it improves at a lower rate (+18% on 8 threads, 95% reads, 100K `key_space`) than `cuckoo_optimistic`, which improves the most (by 60%, to 107.55 Mops/s). `hopscotch` improves at a 39% rate (32.13 Mops/s), only slightly less than for the larger key space. Single-lock data structures improve very little. The overhead of having a second lock is small; the cuckoo map stores 8 bytes of key and value for each bucket, plus a few words of per-bucket metadata (128 bytes/bucket in the baseline code). This structure is just large enough to fit in the L2 cache (12 MB for the M-series P-core), but not in L1 cache (192 KB L1; requires working set of ∼100 KB or less). The optimistic cuckoo reader path is improving cache performance, but that is not our main focus here.

5.6 Radix: concurrency scaling on a fixed structure

Figure 5 depicts the concurrency-discipline effect, demonstrating the shared-prefix performance with `prefix_count=100`, 95% reads. `radix_tree_coarse` performance drops from 21.48 Mops/s generated by 1 thread to only 6.08 Mops/s generated by 8 threads. This anti-scalability comes from 8 threads contending for a single mutex lock, resulting in both serialization overhead and wake-up overhead. `radix_tree_fine`, on the other hand, performance increases from 21.46 Mops/s with 1 thread to 81.86 Mops/s with 8 threads for a 3.81× speedup. The primary reason for the improved concurrency is the synchronization method, and both variants of radix tree are based on a 256-way trie.

The production radix tree written in SGLang's Python implementation is GIL-bound to about the performance of one core for CPU-intensive workloads. Here we present a C++-based replacement called `radix_tree_fine` which reaches a throughput of approximately 3.8× on 8 cores,

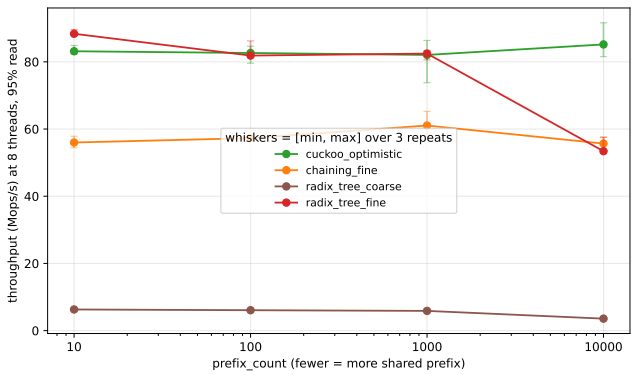

Figure 6: Throughput as prefix sharing increases (8 threads, 95% reads, 100K in universe). `radix_tree_fine` wins at high sharing (88.36 Mops/s, `prefix_count=10`), stays competitive with `cuckoo_optimistic` around the middle of the sweep, and then loses as sharing decreases (trie gets wider, top-level cache locality goes away).

representing an estimate of the upper limit of what a C++ replacement could achieve under similar semantics.

5.7 Radix: sharing-regime sweep

We repeat the experiment of Figure 6, but this time fixing the key universe at 100K and using 95% reads. We vary `prefix_count` $\{10, 100, 1000, 10000\}$ with 8 threads. `radix_tree_fine` wins at `prefix_count=10` (88.36 Mops/s), keeps pace with `cuckoo_optimistic` at `prefix_count`$\in \{100, 1000\}$ (81.86 and 82.45 Mops/s), and drops off sharply at `prefix_count=10000` (53.41 Mops/s) as the radix tree gets wider and fewer top-level nodes remain in cache. `radix_tree_coarse` remains at 3.6–6.3 Mops/s — mutex-bound, not workload-bound. Radix does not necessarily worsen relative to the best hash table; it wins when sharing is concentrated enough to keep the upper trie in cache, matches when sharing is moderate, and loses when every key has a unique prefix.

5.8 Summary

Three results stand out. First, the synthetic winner is not universal: the LLM-derived trace overturns the uniform/Zipfian hash-table ordering once `cuckoo_optimistic`'s displacement path and `hopscotch`'s writer mutex dominate. Second, `radix_tree_fine` matches or beats the best hash table when prefix sharing is concentrated, and degrades when sharing thins. Third, fixing the trie and varying only synchronization yields a $\sim$13.5$\times$ coarse-to-fine radix gap at 8 threads — concurrency discipline alone can dominate structure.

6 Discussion

6.1 What actually determines the winner

Four forces compose in order. (1) A globally serialized path caps throughput: single-lock hash designs stay near $\sim$10 Mops/s at 16 threads, and `radix_tree_coarse` hits a similar ceiling at 8 threads on shared-prefix. (2) Reader concurrency: version-validated reads win on read-heavy synthetics; the lock-free radix walk wins on shared-prefix when sharing is high. (3) Those reader wins hold only while the writer path stays hidden; $\sim$40% writes on the trace expose `cuckoo_optimistic`'s global displacement mutex, `hopscotch`'s serialized writers, and `cuckoo_striped`'s full-stripe displacement. (4) Radix's structural advantage is real but conditional: it needs sufficient prefix sharing *and* synchronization that does not erase the shared-path benefit.

6.2 Implications for LLM serving systems

Two implications follow. First, choosing a prefix-cache index from uniform or Zipfian read-heavy benchmarks alone hides the write path that eviction and new prefixes expose in deployment. Second, "hash vs. radix" is incomplete without the sync strategy: it couples a data-structure choice with a concurrency-control choice; in our measurements the latter can move throughput by more than an order of magnitude on radix alone.

6.3 Broader lesson

Concurrently implementing data structures also means that one has to concurrently test them on workloads which reflect the concurrent usage scenarios that the data structure will be used for. Workloads make a difference; structures ranked highest on one workload may be ranked worst on another. But even beyond that, structures and concurrency control are not independent design parameters: fixing one of them and varying the other will yield misleading rankings in both directions.

7 Limitations

7.1 Implementation limits

"Optimistic" hash designs here are version-validated readers, not lock-free writers: inserts and removes still take writer locks; `cuckoo_optimistic` additionally serializes on a global mutex for displacement; `hopscotch` serializes *all* writers on one mutex. Tables are fixed-size with no resizing; insert success at high load is not guaranteed (see `insert_successes`). `cuckoo_optimistic` uses conservative one-hop displacement to simplify correctness. `radix_tree_fine` still mutex-protects leaf values; fully lock-free ART [5] or non-blocking tries [6] would remove that lock.

7.2 Memory-management limits

Removal in `radix_tree_fine` clears `has_value` but does not reclaim nodes, so memory grows monotonically over a run. Long-lived serving workloads could benefit from SMR-, RCU-, or hazard-pointer-based reclamation. The radix-based code is also an uncompressed byte-level trie; we require path compression for a memory-competitive implementation. We defer path compression.

7.3 Evaluation limits

This test is single-machine and single-NUMA-node. The offline chat data (ShareGPT) has different characteristics to live serving of the same data. Also the timings between requests are somewhat different. The current use of `uint64` keys means we could next move to variable-length token array keys.

7.4 Tooling and verification limits

ThreadSanitizer reports optimistic-reader patterns and the lock-free radix walk as races by construction, so TSan runs use only the fully lock-based subset. Latency below the histogram bin width (64 ns in the sub-μs zone) is not resolvable.

8 Related Work

Prior work supplies concurrent hash tables, tries, and LLM-serving systems as separate ingredients. This paper compares representative designs under one interface and one prefix-cache-shaped workload.

8.1 Concurrent hash tables

Cuckoo hashing was first described by Pagh and Rodler [7]. Later hopscotch hashing was described by Herlihy, Shavit, and Tzafrir [8]. Variants that combine concurrent cuckoo hashing with versioned reads have been described in MemC3 by Fan et al. [9]. In a recent survey, Maier, Sanders, and Dementiev discuss the design choices for concurrent hash tables [10]. All of the hash variants from Section 3 are implemented in a common interface and supported by a common harness in this paper.

8.2 Concurrent tries and radix structures

ART [5] is the reference adaptive radix tree for main-memory databases, and Oshman and Shavit's SkipTrie [6] is representative of low-depth concurrent tries with non-blocking operations. Our work differs in that the comparison target is concurrent hash tables under a workload shaped by prefix caching rather than by database keys.

8.3 LLM serving context

vLLM and PagedAttention [3] motivate hash-table-style indexing for KV blocks; SGLang and RadixAttention [4] motivate radix-tree indexing to exploit shared prefixes. Those systems establish the practical question but evaluate different designs in different codebases. We provide a controlled comparison under a unified interface, harness, and workloads.

9 Conclusion

Hash tables. Under uniform and Zipfian read-heavy mixes, `cuckoo_optimistic` leads `chaining_fine`. Under our ShareGPT-derived trace, the ordering inverts: `chaining_fine` overtakes `cuckoo_optimistic` once ∼40% writes make cuckoo's global displacement mutex dominate. Synthetic hash-table rankings alone are not enough to choose a prefix-cache index.

Radix trees. `radix_tree_fine` matches or beats the best hash table when prefix sharing is high, and degrades when sharing thins. Holding the trie fixed, coarse vs. fine synchronization differs by ∼13.5× at 8 threads, so concurrency discipline is not a footnote.

Takeaway. Structure and synchronization must be chosen together; benchmark realism matters. Future work includes production features (longest-prefix match, integrated eviction, safe reclamation under concurrency) beyond this study's simplified keys and arena-style radix memory.

References

[1] D. Patterson. (2022, Feb.) Good news about the carbon footprint of machine learning training. Google Research. [Online]. Available: https://research.google/blog/good-news-about-the-carbon-footprint-of-machine-learning-training/

[2] N. Minaie, A. K. Lokanatha, M. Hsieh, N. Sendas, R. Ramesha, S. Sachdev, P. K. Sunder, and D. Sury. (2020, Sep.) Ensure efficient compute resources on amazon sagemaker. AWS Blogs. Reviewed and updated for accuracy in November 2023. [Online]. Available: https://aws.amazon.com/blogs/machine-learning/ensure-efficient-compute-resources-on-amazon-sagemaker/

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," *arXiv preprint arXiv:2309.06180*, 2023.

[4] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "Sglang: Efficient execution of structured language model programs," *arXiv preprint arXiv:2312.07104*, 2024.

[5] V. Leis, A. Kemper, and T. Neumann, "The adaptive radix tree: ARTful indexing for main-memory databases," in *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 38–49.

[6] R. Oshman and N. Shavit, "The skiptrie: Low-depth concurrent search without rebalancing," in *Proceedings of the 2013 ACM Symposium on Principles of Distributed Computing (PODC)*, 2013, pp. 23–32.

[7] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[8] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Distributed Computing*, ser. Lecture Notes in Computer Science, vol. 5218. Springer, 2008, pp. 350–364.

[9] B. Fan, D. G. Andersen, and M. Kaminsky, "MemC3: Compact and concurrent memcache with dumber caching and smarter hashing," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, 2013, pp. 371–384.

[10] T. Maier, P. Sanders, and R. Dementiev, "Concurrent hash tables: Fast and general?(!)," *CoRR*, vol. abs/1601.04017, 2016.